INT315: CLUSTER COMPUTING

PROJECT REPORT

(Project Semester August-January 2026)

Monthly Internet Usage Prediction Analysis



Discipline of CSE
Computer Science & Engineering
Lovely Professional University, Phagwara

Submitted to:
Richa (32366)
Assistant Professor
CSE DEPARMENT

Submitted by:
Putta Amulya
12322249 (65)
K2P23FH

TABLE OF CONTENTS

S.No	Title	Page No.
1	Title Page	1
2	Table of Contents	2
3	Introduction	3
4	About Dataset	4
5	Source Code	6
6	Screenshots of Source Code	10
7	Output Screenshots	11
8	Dashboard Screenshot	13

1. Introduction

Internet traffic keeps growing, and network operators need to understand when and how much capacity is used so that they can plan their networks and spot unusual demand. Predicting data usage from past traffic is a common way of supporting that planning. This project builds a small, complete pipeline that loads real network operator traffic measurements, turns them into useful features, trains a model to predict internet usage, and presents the results in a dashboard.

1.1 Objectives

The project covers the following tasks:

- Load a dataset of internet usage patterns using PySpark.
- Create new features, such as peak and off-peak hours.
- Train a Linear Regression model to predict internet data usage.
- Evaluate the model using Mean Absolute Error (MAE).
- Show trends in internet consumption over time, using charts and a dashboard.

1.2 Tools and technologies

Tool	Purpose
Python 3.11.9	Programming language used for the whole pipeline
Apache Spark / PySpark 3.5.0	Distributed data loading, feature engineering and model training (Spark MLlib)
JupyterLab	Interactive notebook used to write and run the code step by step
Pandas and Matplotlib	Summary tables and trend charts
Microsoft Power BI Desktop	Interactive dashboard built from the exported CSV results

1.3 Approach in brief

The raw traffic readings are recorded every five minutes. They are first loaded into a Spark DataFrame and converted into timestamps. Features for the hour of the day, the day of the week and a peak/off-peak flag are then added, and the readings are summed into hourly usage for each site. The data is split by time, with the earlier 80% of days used for training and the later 20% for testing. A Linear Regression model is trained and evaluated with MAE, and the results are exported as CSV files that feed the Power BI dashboard.

The project title refers to monthly usage, but the dataset covers only about 37 days, which is too short to build a meaningful monthly model. The model therefore predicts hourly usage, and the results are summarised as daily totals to show consumption trends.

2. About the Dataset

2.1 Source

The project uses the Network Operator KPIs time series dataset, published on Zenodo (record 8147768). It contains Key Performance Indicator (KPI) time series measured at sites of a telecommunications network operator, and it is intended for research on time series analysis and anomaly detection.

Link: https://zenodo.org/records/8147768?utm_source=chatgpt.com

2.2 What the dataset contains

The downloaded archive is organised into folders of plain text files, together with description files:

- **data_series:** the main set of KPI series, one text file per series (s1.txt, s2.txt, and so on). The copy used here contains 48 files and about 515,000 rows in total.
- **data_real:** a smaller set of series (r1 to r14) that carry labelled anomalies, listed in data_real_incidents.txt. It is meant for anomaly detection and was not used in this project.
- **Information files and README:** these describe the dataset and map each series to its KPI type.

2.3 File format

The files have no header row. Each line holds two values separated by a space:

Column	Meaning	Example
1	Seconds since the start of the series (starts at 0, in steps of 300 s = 5 minutes)	300
2	KPI value, scaled to the range 0 to 1000	482.26

Because there is no header, the code assigns the column names itself (t_sec and traffic) when the files are read.

2.4 KPI types

The series are grouped by the kind of measurement they represent:

Series	KPI type	What it measures
s1 to s10	Internet	Total internet traffic at a network site
s11 to s30	Sessions	Number of active client sessions

Series	KPI type	What it measures
s31 to s40	VPN	VPN traffic
s41 to s50	Downstream	Download-direction traffic

The project uses the Internet series, s1 to s10, because they are the closest match to internet data usage. The traffic values are measured as a rate, not as the amount of data used by each customer.

2.5 Data used in this project

Property	Value
Series used	s1 to s10 (internet traffic), one per site
Readings per series	10,738, one every 5 minutes
Total raw rows	107,380
Time span	About 37 days
Value range	0 to 1000 (scaled)
Hourly rows after aggregation	8,950 (10 sites over about 895 hours)
Training rows / test rows	7,200 / 1,750 (time-based 80% / 20% split)

2.6 Features created

Feature	Description
hour	Hour of the day (0 to 23)
dow	Day of the week (0 to 6)
is_peak	1 for hours 18:00 to 23:59 (peak), 0 otherwise (off-peak)
day_idx	Day number since the start of the data, so the model can learn a trend
site	Which site the reading comes from (used in the improved model)
usage	Target: the sum of the 5-minute readings within an hour for a site

2.7 Limitations of the data

- The values are scaled between 0 and 1000, so they are relative traffic units and not gigabytes.
- The timestamps are anonymised and start at 0. A placeholder start date (1 January 2023) was assigned, so the hour of the day and the day of the week are relative and not real calendar values.
- The dataset covers only about 37 days, so monthly modelling is not possible and the final day contains only a few hours of data.
- The data shows traffic at network sites, not the data usage of individual subscribers.

3.Source Code

Github link: https://github.com/amulya916/Monthly-internet-usage-prediction-analysis

```
DATA_DIR = "network_operator_KPIs_time_series_dataset/data_series"
INTERNET_SERIES = [f"s{i}" for i in range(1, 11)]
START_DATE = "2023-01-01 00:00:00"
PEAK_HOURS = (18, 23)
OUT_DIR = "output"
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("InternetUsage").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
print("Spark version:", spark.version)
frames = []
for sid in INTERNET_SERIES:
  d = (spark.read.option("sep", " ").option("inferSchema", True)
    .csv(f"{DATA_DIR}/{sid}.txt").toDF("t_sec", "traffic")
    .withColumn("site", F.lit(sid)))
  frames.append(d)
raw = frames[0]
for f in frames[1:]:
```

```
    raw = raw.unionByName(f)
print("Rows:", raw.count())
raw.show(5)
raw.printSchema()
raw = (raw.dropna()
       .withColumn("ts", (F.unix_timestamp(F.lit(START_DATE)) +
F.col("t_sec")).cast("timestamp")))
raw.show(5)
feat = (raw.withColumn("date", F.to_date("ts"))
        .withColumn("hour", F.hour("ts"))
        .withColumn("dow", F.dayofweek("ts") - 1)
        .withColumn("is_peak", F.when(F.col("hour").between(*PEAK_HOURS), 1).otherwise(0)))
feat.select("ts", "traffic", "hour", "dow", "is_peak").show(5)
hourly = (feat.groupBy("site", "date", "hour", "dow", "is_peak")
          .agg(F.sum("traffic").alias("usage"))
          .withColumn("day_idx", F.datediff("date", F.lit(START_DATE.split()[0]).cast("date"))))
print("Hourly rows:", hourly.count())
hourly.orderBy("site", "date", "hour").show(5)
max_day = hourly.agg(F.max("day_idx")).first()[0]
cut = int(max_day * 0.8)
train = hourly.filter(F.col("day_idx") <= cut)
test = hourly.filter(F.col("day_idx") > cut)
print("Train rows:", train.count(), "| Test rows:", test.count())
from pyspark.ml import Pipeline
from pyspark.ml.feature import OneHotEncoder, VectorAssembler
from pyspark.ml.regression import LinearRegression


enc = OneHotEncoder(inputCols=["hour", "dow"], outputCols=["hour_oh", "dow_oh"])
asm = VectorAssembler(inputCols=["hour_oh", "dow_oh", "is_peak", "day_idx"],
outputCol="features")
lr = LinearRegression(featuresCol="features", labelCol="usage", predictionCol="prediction")
model = Pipeline(stages=[enc, asm, lr]).fit(train)
```

```
pred = model.transform(test)

pred.select("site", "date", "hour", "usage", "prediction").show(5)

from pyspark.ml.evaluation import RegressionEvaluator

mae = RegressionEvaluator(labelCol="usage", predictionCol="prediction",
metricName="mae").evaluate(pred)

avg_usage = test.agg(F.avg("usage")).first()[0]

print(f"Test MAE: {mae:.2f}")

print(f"Average usage in test set: {avg_usage:.2f}")

from pyspark.ml.feature import StringIndexer


idx = StringIndexer(inputCol="site", outputCol="site_idx")

enc2 = OneHotEncoder(inputCols=["hour", "dow", "site_idx"], outputCols=["hour_oh", "dow_oh",
"site_oh"])

asm2 = VectorAssembler(inputCols=["hour_oh", "dow_oh", "site_oh", "is_peak", "day_idx"],
outputCol="features")

model2 = Pipeline(stages=[idx, enc2, asm2, lr]).fit(train)

pred2 = model2.transform(test)


mae2 = RegressionEvaluator(labelCol="usage", predictionCol="prediction",
metricName="mae").evaluate(pred2)

print(f"MAE without site: {mae:.2f} ({100*mae/avg_usage:.1f}%)")

print(f"MAE with site:    {mae2:.2f} ({100*mae2/avg_usage:.1f}%)")

import matplotlib.pyplot as plt

pred = pred2

mae = mae2


complete = hourly.groupBy("date").count().filter(F.col("count") == 240).select("date")


daily_pd = (hourly.join(complete, "date")

        .groupBy("date").agg(F.sum("usage").alias("total_usage"))

        .orderBy("date").toPandas())

hour_pd =
hourly.groupBy("hour").agg(F.avg("usage").alias("avg_usage")).orderBy("hour").toPandas()
```

```
avp_pd = (pred.join(complete, "date")
        .groupBy("date").agg(F.sum("usage").alias("actual"),
F.sum("prediction").alias("predicted"))
        .orderBy("date").toPandas())


fig, ax = plt.subplots(1, 3, figsize=(18, 4))
ax[0].plot(daily_pd["date"], daily_pd["total_usage"]); ax[0].set_title("Daily total usage")
ax[1].bar(hour_pd["hour"], hour_pd["avg_usage"]); ax[1].set_title("Average usage by hour")
ax[2].plot(avp_pd["date"], avp_pd["actual"], label="Actual")
ax[2].plot(avp_pd["date"], avp_pd["predicted"], label="Predicted"); ax[2].legend();
ax[2].set_title("Actual vs predicted (test, complete days)")
for a in ax: a.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
import os, pandas as pd
os.makedirs(OUT_DIR, exist_ok=True)
daily_pd.to_csv(f"{OUT_DIR}/daily_trend.csv", index=False)
hour_pd.to_csv(f"{OUT_DIR}/hourly_pattern.csv", index=False)
avp_pd.to_csv(f"{OUT_DIR}/actual_vs_predicted.csv", index=False)
(hourly.groupBy("is_peak").agg(F.sum("usage").alias("total_usage"))
    .withColumn("period", F.when(F.col("is_peak") == 1, "Peak").otherwise("Off-peak"))
    .toPandas().to_csv(f"{OUT_DIR}/peak_vs_offpeak.csv", index=False))
pd.DataFrame({"total_traffic": [hourly.agg(F.sum("usage")).first()[0]], "mae":
[mae]}).to_csv(f"{OUT_DIR}/kpis.csv", index=False)
print("MAE exported:", round(mae, 2))
print(os.listdir(OUT_DIR))
```

4. Screenshots of Source Code

```
DATA_DIR = "network_operator_KPIs_time_series_dataset/data_series"
INTERNET_SERIES = [f"s{i}" for i in range(1, 11)]
START_DATE = "2023-01-01 00:00:00"
PEAK_HOURS = (18, 23)
OUT_DIR = "output"
```

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("InternetUsage").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
print("Spark version:", spark.version)
```

```
Spark version: 3.5.0
```

```
frames = []
for sid in INTERNET_SERIES:
    d = (spark.read.option("sep", " ").option("inferSchema", True)
         .csv(f"{DATA_DIR}/{sid}.txt").toDF("t_sec", "traffic")
         .withColumn("site", F.lit(sid)))
    frames.append(d)
raw = frames[0]
for f in frames[1:]:
    raw = raw.unionByName(f)
print("Rows:", raw.count())
raw.show(5)
raw.printSchema()
```

```
raw = (raw.dropna()
          .withColumn("ts", (F.unix_timestamp(F.lit(START_DATE)) + F.col("t_sec")).cast("timestamp")))
raw.show(5)
```

```
feat = (raw.withColumn("date", F.to_date("ts"))
           .withColumn("hour", F.hour("ts"))
           .withColumn("dow", F.dayofweek("ts") - 1)
           .withColumn("is_peak", F.when(F.col("hour").between(*PEAK_HOURS), 1).otherwise(0)))
feat.select("ts", "traffic", "hour", "dow", "is_peak").show(5)
```

```
hourly = (feat.groupBy("site", "date", "hour", "dow", "is_peak")
              .agg(F.sum("traffic").alias("usage"))
              .withColumn("day_idx", F.datediff("date", F.lit(START_DATE.split()[0]).cast("date"))))
print("Hourly rows:", hourly.count())
hourly.orderBy("site", "date", "hour").show(5)
```

```
max_day = hourly.agg(F.max("day_idx")).first()[0]
cut = int(max_day * 0.8)
train = hourly.filter(F.col("day_idx") <= cut)
test = hourly.filter(F.col("day_idx") > cut)
print("Train rows:", train.count(), "| Test rows:", test.count())
```

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import OneHotEncoder, VectorAssembler
from pyspark.ml.regression import LinearRegression

enc = OneHotEncoder(inputCols=["hour", "dow"], outputCols=["hour_oh", "dow_oh"])
asm = VectorAssembler(inputCols=["hour_oh", "dow_oh", "is_peak", "day_idx"], outputCol="features")
lr = LinearRegression(featuresCol="features", labelCol="usage", predictionCol="prediction")
model = Pipeline(stages=[enc, asm, lr]).fit(train)
pred = model.transform(test)
pred.select("site", "date", "hour", "usage", "prediction").show(5)
```

```
from pyspark.ml.evaluation import RegressionEvaluator
mae = RegressionEvaluator(labelCol="usage", predictionCol="prediction", metricName="mae").evaluate(pred)
avg_usage = test.agg(F.avg("usage")).first()[0]
print(f"Test MAE: {mae:.2f}")
print(f"Average usage in test set: {avg_usage:.2f}")
```

```python
from pyspark.ml.feature import StringIndexer

idx = StringIndexer(inputCol="site", outputCol="site_idx")
enc2 = OneHotEncoder(inputCols=["hour", "dow", "site_idx"], outputCols=["hour_oh", "dow_oh", "site_oh"])
asm2 = VectorAssembler(inputCols=["hour_oh", "dow_oh", "site_oh", "is_peak", "day_idx"], outputCol="features")
model2 = Pipeline(stages=[idx, enc2, asm2, lr]).fit(train)
pred2 = model2.transform(test)

mae2 = RegressionEvaluator(labelCol="usage", predictionCol="prediction", metricName="mae").evaluate(pred2)
print(f"MAE without site: {mae:.2f} ({100*mae/avg_usage:.1f}%)")
print(f"MAE with site:    {mae2:.2f} ({100*mae2/avg_usage:.1f}%)")
```

```python
import matplotlib.pyplot as plt
pred = pred2
mae = mae2

complete = hourly.groupBy("date").count().filter(F.col("count") == 240).select("date")

daily_pd = (hourly.join(complete, "date")
                  .groupBy("date").agg(F.sum("usage").alias("total_usage"))
                  .orderBy("date").toPandas())
hour_pd = hourly.groupBy("hour").agg(F.avg("usage").alias("avg_usage")).orderBy("hour").toPandas()
avp_pd = (pred.join(complete, "date")
              .groupBy("date").agg(F.sum("usage").alias("actual"), F.sum("prediction").alias("predicted"))
              .orderBy("date").toPandas())

fig, ax = plt.subplots(1, 3, figsize=(18, 4))
ax[0].plot(daily_pd["date"], daily_pd["total_usage"]); ax[0].set_title("Daily total usage")
ax[1].bar(hour_pd["hour"], hour_pd["avg_usage"]); ax[1].set_title("Average usage by hour")
ax[2].plot(avp_pd["date"], avp_pd["actual"], label="Actual")
ax[2].plot(avp_pd["date"], avp_pd["predicted"], label="Predicted"); ax[2].legend(); ax[2].set_title("Actual vs predicted (test, complete days)")
for a in ax: a.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
```

```python
import os, pandas as pd
os.makedirs(OUT_DIR, exist_ok=True)
daily_pd.to_csv(f"{OUT_DIR}/daily_trend.csv", index=False)
hour_pd.to_csv(f"{OUT_DIR}/hourly_pattern.csv", index=False)
avp_pd.to_csv(f"{OUT_DIR}/actual_vs_predicted.csv", index=False)
(hourly.groupBy("is_peak").agg(F.sum("usage").alias("total_usage"))
       .withColumn("period", F.when(F.col("is_peak") == 1, "Peak").otherwise("Off-peak"))
       .toPandas().to_csv(f"{OUT_DIR}/peak_vs_offpeak.csv", index=False))
pd.DataFrame({"total_traffic": [hourly.agg(F.sum("usage")).first()[0]], "mae": [mae]}).to_csv(f"{OUT_DIR}/kpis.csv", index=False)
print("MAE exported:", round(mae, 2))
print(os.listdir(OUT_DIR))
```

5. Output Screenshots

```
Rows: 107380
+-----+------------------+----+
|t_sec|           traffic|site|
+-----+------------------+----+
|    0|309.63413722707816|  s1|
|  300|300.17454480088946|  s1|
|  600|289.49900721247286|  s1|
|  900|280.83066222579055|  s1|
| 1200| 273.6886439905293|  s1|
+-----+------------------+----+
only showing top 5 rows

root
 |-- t_sec: integer (nullable = true)
 |-- traffic: double (nullable = true)
 |-- site: string (nullable = false)
```

```
+-----+------------------+----+-------------------+
|t_sec|           traffic|site|                 ts|
+-----+------------------+----+-------------------+
|    0|309.63413722707816|  s1|2023-01-01 00:00:00|
|  300|300.17454480088946|  s1|2023-01-01 00:05:00|
|  600|289.49900721247286|  s1|2023-01-01 00:10:00|
|  900|280.83066222579055|  s1|2023-01-01 00:15:00|
| 1200| 273.6886439905293|  s1|2023-01-01 00:20:00|
+-----+------------------+----+-------------------+
only showing top 5 rows
```

```
+-------------------+------------------+----+---+-------+
|                 ts|           traffic|hour|dow|is_peak|
+-------------------+------------------+----+---+-------+
|2023-01-01 00:00:00|309.63413722707816|   0|  0|      0|
|2023-01-01 00:05:00|300.17454480088946|   0|  0|      0|
|2023-01-01 00:10:00|289.49900721247286|   0|  0|      0|
|2023-01-01 00:15:00|280.83066222579055|   0|  0|      0|
|2023-01-01 00:20:00| 273.6886439905293|   0|  0|      0|
+-------------------+------------------+----+---+-------+
only showing top 5 rows
```

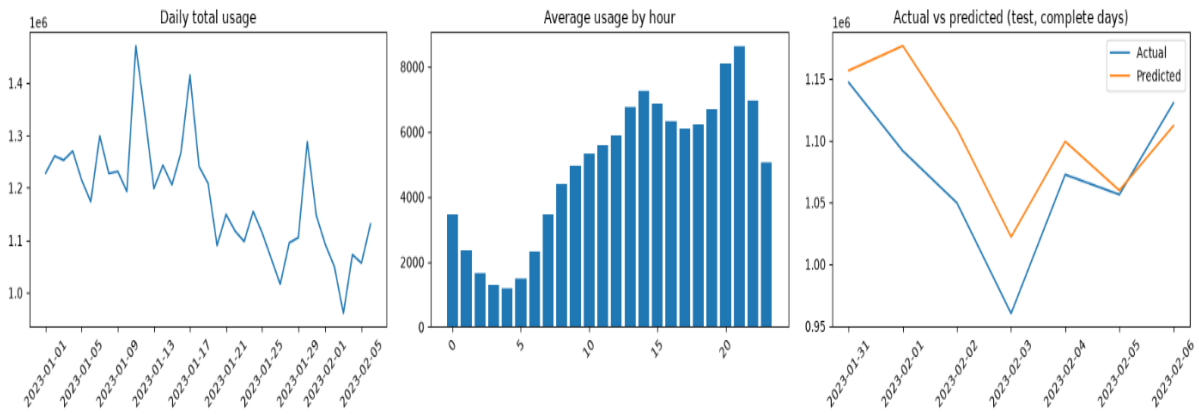
```
Hourly rows: 8950
+----+----------+----+---+-------+------------------+-------+
|site|      date|hour|dow|is_peak|             usage|day_idx|
+----+----------+----+---+-------+------------------+-------+
|  s1|2023-01-01|   0|  0|      0| 3128.125888751178|      0|
|  s1|2023-01-01|   1|  0|      0| 2078.628467101514|      0|
|  s1|2023-01-01|   2|  0|      0|1457.3177036692196|      0|
|  s1|2023-01-01|   3|  0|      0|1088.7660112765204|      0|
|  s1|2023-01-01|   4|  0|      0|1005.1197603974152|      0|
+----+----------+----+---+-------+------------------+-------+
only showing top 5 rows
```

```
Train rows: 7200 | Test rows: 1750
```

```
+----+----------+----+------------------+------------------+
|site|      date|hour|             usage|        prediction|
+----+----------+----+------------------+------------------+
|  s1|2023-02-04|   2|1468.1499014520534|1217.8839047882138|
|  s1|2023-02-05|  16| 5766.586494620438|5838.0811253419815|
|  s1|2023-02-06|  18|5698.5661047832045| 5963.27363168354|
|  s1|2023-02-02|  17| 5035.972667278587| 5822.381671144431|
|  s1|2023-02-04|   4| 903.6058984421763| 744.5252307414958|
+----+----------+----+------------------+------------------+
only showing top 5 rows
```

```
Test MAE: 558.55
Average usage in test set: 4363.07
```

```
MAE without site: 558.55 (12.8%)
MAE with site:    459.89 (10.5%)
```



6. Dashboard Screenshot

